

In other words, $\mathbf{x}_s$ has the same distribution if one (1) directly samples it by interpolating $\mathbf{x}$ and $\mathbf{x}_t$ and (2) first samples any $\mathbf{x}_r$ (given $\mathbf{x}$ and $\mathbf{x}_t$) and then samples $\mathbf{x}_s$ (given $\mathbf{x}$ and $\mathbf{x}_r$). Furthermore, self-consistency implies marginal preservation (Lemma 5).

DDIM interpolant. Denoising Diffusion Implicit Models (Song et al., 2020a) was introduced as a fast ODE sampler for diffusion models, defined as

$$\text{DDIM}(\mathbf{x}_t, \mathbf{x}, s, t) = \left(\alpha_s - \frac{\sigma_s}{\sigma_t} \alpha_t \right) \mathbf{x} + \frac{\sigma_s}{\sigma_t} \mathbf{x}_t \quad (11)$$

and sample $\mathbf{x}_s = \text{DDIM}(\mathbf{x}_t, \mathbb{E}_{\mathbf{x}}[\mathbf{x}|\mathbf{x}_t], s, t)$ can be drawn when $\mathbb{E}_{\mathbf{x}}[\mathbf{x}|\mathbf{x}_t]$ is approximated by a network. We show in Appendix C.1 that DDIM as an interpolant, i.e. $\gamma_{s|t} \equiv 0$ and $\mathbf{I}_{s|t}(\mathbf{x}, \mathbf{x}_t) = \text{DDIM}(\mathbf{x}_t, \mathbf{x}, s, t)$, is self-consistent. Moreover, with deterministic interpolants such as DDIM, there exists a deterministic minimizer $p_{s|t}^\theta(\mathbf{x}|\mathbf{x}_t)$ of Eq. (7).

Proposition 1. (Informal) If $\gamma_{s|t} \equiv 0$ and $\mathbf{I}_{s|t}(\mathbf{x}, \mathbf{x}_t)$ satisfies mild assumptions, there exists a deterministic $p_{s|t}^\theta(\mathbf{x}|\mathbf{x}_t)$ that attains 0 loss for Eq. (7).

See Appendix B.6 for formal statement and proof. This allows us to define $p_{s|t}^\theta(\mathbf{x}|\mathbf{x}_t) = \delta(\mathbf{x} - \mathbf{g}_\theta(\mathbf{x}_t, s, t))$ for a neural network $\mathbf{g}_\theta(\mathbf{x}_t, s, t)$ with parameter θ by default.

Eliminating stochasticity. We use DDIM interpolant, deterministic model, and prior $p(\epsilon) = \mathcal{N}(0, \sigma_d^2 I)$ where σ_d is the data standard deviation (Lu & Song, 2024). As a result, one can draw $\mathbf{x}_s$ from model via $\mathbf{x}_s = \mathbf{f}_{s,t}^\theta(\mathbf{x}_t) := \text{DDIM}(\mathbf{x}_t, \mathbf{g}_\theta(\mathbf{x}_t, s, t), s, t)$ where $\mathbf{x}_t \sim q_t(\mathbf{x}_t)$.

Re-using $\mathbf{x}_t$ for $\mathbf{x}_r$. Inspecting Eq. (8) and (6), one requires $\mathbf{x}_r \sim q_r(\mathbf{x}_r)$ to generate samples from the target distribution. Instead of sampling $\mathbf{x}_r$ given a new $(\mathbf{x}, \epsilon)$ pair, we can reduce variance by reusing $\mathbf{x}_t$ and $\mathbf{x}$ such that $\mathbf{x}_r = \text{DDIM}(\mathbf{x}_t, \mathbf{x}, r, t)$. This is justified because $\mathbf{x}_r$ derived from $\mathbf{x}_t$ preserves the marginal distribution $q_r(\mathbf{x}_r)$ (see Appendix C.2).

Stop gradient. We set n to optimization step number, i.e. advancing from $n-1$ to n is a single optimizer step where θ_n is initialized from θ_{n-1} . Equivalently, we can omit n from θ_n and write θ_{n-1} as the stop-gradient parameter θ^- .

Simplified objective. Let $\mathbf{x}_t, \mathbf{x}_t'$ be i.i.d. random variables from $q_t(\mathbf{x}_t)$ and $\mathbf{x}_r, \mathbf{x}_r'$ are variables obtained by reusing $\mathbf{x}_t, \mathbf{x}_t'$ respectively, the training objective can be derived from the MMD definition in Eq. (1) (see Appendix C.3) as

$$\mathcal{L}_{\text{IMM}}(\theta) = \mathbb{E}_{\mathbf{x}_t, \mathbf{x}_t', \mathbf{x}_r, \mathbf{x}_r', s, t} \left[w(s, t) \left[k(\mathbf{y}_{s,t}, \mathbf{y}_{s,t}') - k(\mathbf{y}_{s,r}, \mathbf{y}_{s,r}') - k(\mathbf{y}_{s,t}, \mathbf{y}_{s,r}) - k(\mathbf{y}_{s,t}', \mathbf{y}_{s,r}') \right] \right] \quad (12)$$

where $\mathbf{y}_{s,t} = \mathbf{f}_{s,t}^\theta(\mathbf{x}_t)$, $\mathbf{y}_{s,t}' = \mathbf{f}_{s,t}^\theta(\mathbf{x}_t')$, $\mathbf{y}_{s,r} = \mathbf{f}_{s,r}^{\theta^-}(\mathbf{x}_r)$, $\mathbf{y}_{s,r}' = \mathbf{f}_{s,r}^{\theta^-}(\mathbf{x}_r')$, $k(\cdot, \cdot)$ is a kernel function, and $w(s, t)$ is a prior weighting function.

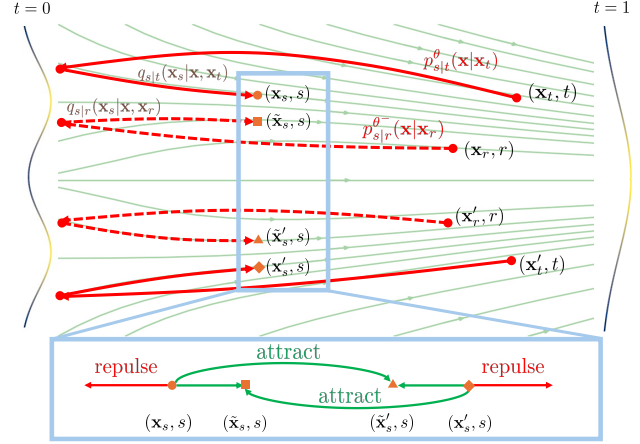


Figure 3. With self-consistent interpolants, IMM uses M particle samples ($M = 2$ is shown above) for moment matching. Samples from $p_{s|t}^\theta(\mathbf{x}_s)$ are obtained by drawing from $p_{s|t}^\theta(\mathbf{x}|\mathbf{x}_t)$ followed by $q_{s|t}(\mathbf{x}_s|\mathbf{x}, \mathbf{x}_t)$. Solid and dashed red lines indicate sampling with and without gradient propagation respectively. After M samples are drawn, sample $\mathbf{x}_s$ is repulsed by $\mathbf{x}_s'$ and attracted towards samples of $\tilde{\mathbf{x}}_s$ and $\tilde{\mathbf{x}}_s'$ through kernel function $k(\cdot, \cdot)$.

An empirical estimate of the above objective uses M particle samples to approximate each distribution indexed by t . In practice, we divide a batch of model output with size B into B/M groups within which share the same (s, t) sample, and the objective is approximated by instantiating B/M number of $M \times M$ matrices. Note that the number of model passes does not change with respect to M (see Appendix C.4). A $M = 2$ version is visualized in Figure 3 and a simplified training algorithm is shown in Algorithm 1. A full training algorithm is shown in Appendix D.

4.2. Other Implementation Choices

We defer detailed analysis of each decision to Appendix C.

Flow trajectories. We investigate the two most used flow trajectories (Nichol & Dhariwal, 2021; Lipman et al., 2022),

- **Cosine.** $\alpha_t = \cos(\frac{1}{2}\pi t)$, $\sigma_t = \sin(\frac{1}{2}\pi t)$.
- **OT-FM.** $\alpha_t = 1 - t$, $\sigma_t = t$.

Network $\mathbf{g}_\theta(\mathbf{x}_t, s, t)$. We set $\mathbf{g}_\theta(\mathbf{x}_t, s, t) = c_{\text{skip}}(t)\mathbf{x}_t + c_{\text{out}}(t)\mathbf{G}_\theta(c_{\text{in}}(t)\mathbf{x}_t, c_{\text{noise}}(s), c_{\text{noise}}(t))$ with a neural network $\mathbf{G}_\theta$, following EDM (Karras et al., 2022). For all choices we let $c_{\text{in}}(t) = 1/\sqrt{\alpha_t^2 + \sigma_t^2}/\sigma_d$ (Lu & Song, 2024). Listed below are valid choices for other coefficients.

- **Identity.** $c_{\text{skip}}(t) = 0$, $c_{\text{out}}(t) = 1$.
- **Simple-EDM** (Lu & Song, 2024). $c_{\text{skip}}(t) = \alpha_t/(\alpha_t^2 + \sigma_t^2)$, $c_{\text{out}}(t) = -\sigma_d\sigma_t/\sqrt{\alpha_t^2 + \sigma_t^2}$.
- **Euler-FM.** $c_{\text{skip}}(t) = 1$, $c_{\text{out}}(t) = -t\sigma_d$. This is specific to OT-FM schedule.

We show in Appendix C.5 that $\mathbf{f}_{s,t}^\theta(\mathbf{x}_t)$ similarly follows the

EDM parameterization of the form $\mathbf{f}_{s,t}^\theta(\mathbf{x}_t) = c_{\text{skip}}(s, t)\mathbf{x}_t + c_{\text{out}}(s, t)\mathbf{G}_\theta(c_{\text{in}}(t)\mathbf{x}_t, c_{\text{noise}}(s), c_{\text{noise}}(t))$.

Noise conditioning $c_{\text{noise}}(\cdot)$. We choose $c_{\text{noise}}(t) = ct$ for some constant $c \geq 1$. We find our model convergence relatively insensitive to c but recommend using larger c , e.g. 1000 (Song et al., 2020b; Peebles & Xie, 2023), because it enables sufficient distinction between nearby r and t .

Mapping function $r(s, t)$. We find that $r(s, t)$ via constant decrement in $\eta_t = \sigma_t/\alpha_t$ works well where the decrement is chosen in the form of $(\eta_{\text{max}} - \eta_{\text{min}})/2^k$ for some appropriate k (details in Appendix C.7).

Kernel function. We use time-dependent Laplace kernels of the form $k_{s,t}(x, y) = \exp(-\tilde{w}(s, t) \max(\|x - y\|_2, \epsilon)/D)$ for $x, y \in \mathbb{R}^D$, some $\epsilon > 0$ to avoid undefined gradients, and $\tilde{w}(s, t) = 1/|c_{\text{out}}(s, t)|$. We find Laplace kernels provide better gradient signals than RBF kernels. (see Appendix C.8).

Weighting $w(s, t)$ **and distribution** $p(s, t)$. We follow VDM (Kingma et al., 2021; Kingma & Gao, 2024) and define $p(t) = \mathcal{U}(\epsilon, T)$ and $p(s|t) = \mathcal{U}(\epsilon, t)$ for constants $\epsilon, T \in [0, 1]$. Similarly, weighting is defined as

$$w(s, t) = \frac{1}{2}\sigma(b - \lambda_t) \left(-\frac{d}{dt}\lambda_t \right) \frac{\alpha_t^a}{\alpha_t^2 + \sigma_t^2} \quad (13)$$

where $\sigma(\cdot)$ is sigmoid function, λ_t denotes $\log\text{-SNR}_t$ and $a \in \{1, 2\}$, $\sigma(\cdot)$, $b \in \mathbb{R}$ are constants (see Appendix C.9).

4.3. Sampling

Pushforward sampling. A sample $\mathbf{x}_s$ can be obtained by directly pushing $\mathbf{x}_t \sim q_t(\mathbf{x}_t)$ through $\mathbf{f}_{s,t}^\theta(\mathbf{x}_t)$. This can be iterated for an arbitrary number of steps starting from $\epsilon \sim \mathcal{N}(0, \sigma_d^2 I)$ until $s = 0$. We note that, by definition, one application of $\mathbf{f}_{s,t}^\theta(\mathbf{x}_t)$ is equivalent to one DDIM step using the learned network $\mathbf{g}_\theta(\mathbf{x}_t, s, t)$ as the $\mathbf{x}$ prediction. This sampler can then be viewed as a few-step sampler using DDIM where $\mathbf{g}_\theta(\mathbf{x}_t, s, t)$ outputs a realistic sample $\mathbf{x}$ instead of its expectation $\mathbb{E}_{\mathbf{x}}[\mathbf{x}|\mathbf{x}_t]$ as in diffusion models.

Restart sampling. Similar to Xu et al. (2023); Song et al. (2023), one can introduce stochasticity during sampling by re-noising a sample to a higher noise-level before sampling again. For example, a two-step restart sampler from $\mathbf{x}_t$ requires $s \in (0, t)$ for drawing sample $\hat{\mathbf{x}} = \mathbf{f}_{0,s}^\theta(\mathbf{x}_s)$ where $\mathbf{x}_s \sim q_s(\mathbf{x}_s|\mathbf{f}_{0,t}^\theta(\mathbf{x}_t))$.

Classifier-free guidance. Given a data-label pair $(\mathbf{x}, \mathbf{c})$, during inference time, classifier-free guidance (Ho & Salimans, 2022) with weight w replaces conditional model output $\mathbf{G}_\theta(\mathbf{x}_t, s, t, \mathbf{c})$ by a reweighted model output via

$$w\mathbf{G}_\theta(\mathbf{x}_t, s, t, \mathbf{c}) + (1 - w)\mathbf{G}_\theta(\mathbf{x}_t, s, t, \emptyset) \quad (14)$$

where $\emptyset$ denotes the null-token indicating unconditional out-

Algorithm 1 Training (see Appendix D for full version)

Input: parameter θ , DDIM($\mathbf{x}_t, \mathbf{x}, s, t$), B, M, p
Output: learned θ
while model not converged **do**
 Sample data x , label c , and prior ϵ with batch size B and split into B/M groups. Each group shares a (s, r, t) sample.
 For each group, $x_t \leftarrow \text{DDIM}(\epsilon, x, t, 1)$.
 For each group, $x_r \leftarrow \text{DDIM}(x_t, x, r, t)$.
 For each instance, set $c = \emptyset$ with prob. p .
 Minimize the empirical loss $\hat{\mathcal{L}}_{\text{IMM}}(\theta)$ in Eq. (67).
end while

Algorithm 2 Pushforward Sampling (details in Appendix F)

Input: model $\mathbf{f}^\theta$, $\{t_i\}_{i=0}^N$, $\mathcal{N}(0, \sigma_d^2 I)$, (optional) w
Output: $\mathbf{x}_{t_0}$
Sample $\mathbf{x}_N \sim \mathcal{N}(0, \sigma_d^2 I)$
for $i = N, \dots, 1$ **do**
 $\mathbf{x}_{t_{i-1}} \leftarrow \mathbf{f}_{t_{i-1}, t_i}^\theta(\mathbf{x}_{t_i})$ or $\mathbf{f}_{t_{i-1}, t_i, w}^\theta(\mathbf{x}_{t_i})$.
end for

put. Similarly, we define our guided model as $\mathbf{f}_{s,t,w}^\theta(\mathbf{x}_t) = c_{\text{skip}}(s, t)\mathbf{x}_t + c_{\text{out}}(s, t)\mathbf{G}_\theta^w(\mathbf{x}_t, s, t, \mathbf{c})$ where $\mathbf{G}_\theta^w(\mathbf{x}_t, s, t, \mathbf{c})$ is as defined in Eq. (14) and we drop $c_{\text{in}}(\cdot)$ and $c_{\text{noise}}(\cdot)$ for notational simplicity. We justify this decision in Appendix E. Similar to diffusion models, $\mathbf{c}$ is randomly dropped with probability p during training without special practices.

We present pushforward sampling in Algorithm 2 and detail both samplers in Appendix F.

5. Connection with Prior Works

Our work is closely connected with many prior works. Detailed analysis is found in Appendix G.

Consistency Models. Consistency models (CMs) (Song et al., 2023; Song & Dhariwal, 2023; Lu & Song, 2024) uses a network $\mathbf{g}_\theta(\mathbf{x}_t, t)$ that outputs clean data given noisy input $\mathbf{x}_t$. It requires point-wise consistency $\mathbf{g}_\theta(\mathbf{x}_t, t) = \mathbf{g}_\theta(\mathbf{x}_r, r)$ for any $r < t$ where $\mathbf{x}_r$ is obtained via an ODE solver from $\mathbf{x}_t$ using either pretrained model or groundtruth data. Discrete-time CM must satisfy $\mathbf{g}_\theta(\mathbf{x}_0, 0) = \mathbf{x}_0$ and trains via loss $\mathbb{E}_{\mathbf{x}_t, \mathbf{x}, t}[d(\mathbf{g}_\theta(\mathbf{x}_t, t), \mathbf{g}_\theta(\mathbf{x}_r, r))]$ where $d(\cdot, \cdot)$ is commonly chosen as $\mathcal{L}_2$ or LPIPS (Zhang et al., 2018).

We show in the following Lemma that CM objective with $\mathcal{L}_2$ distance is a single-particle estimate of IMM objective with energy kernel.

Lemma 1. When $\mathbf{x}_t = \mathbf{x}'_t$, $\mathbf{x}_r = \mathbf{x}'_r$, $k(x, y) = -\|x - y\|^2$, and $s > 0$ is a small constant, Eq. (12) reduces to CM loss $\mathbb{E}_{\mathbf{x}_t, \mathbf{x}, t} \left[w(t) \|\mathbf{g}_\theta(\mathbf{x}_t, t) - \mathbf{g}_\theta(\mathbf{x}_r, r)\|^2 \right]$ for some valid mapping $r(t) < t$.

This single-particle estimate ignores the repulsion force

imposed by $k(\cdot, \cdot)$. Energy kernel also only matches the *first* moment, ignoring all higher moments. These decisions can be significant contributors to training instability and performance degradation of CMs.

Improved CMs (Song & Dhariwal, 2023) propose pseudo-huber loss as $d(\cdot, \cdot)$ which we justify in the Lemma below.

Lemma 2. *Negative pseudo-huber loss $k_c(x, y) = c - \sqrt{\|x - y\|^2 + c^2}$ for $c > 0$ is a conditionally positive definite kernel that matches all moments of x and y where weights on higher moments depend on c .*

From a moment-matching perspective, the improved performance is explained by the loss matching all moments of the distributions. In addition to pseudo-huber loss, many other kernels (Laplace, RBF, etc.) are all valid choices in the design space.

We also extend IMM loss to the differential limit by taking $r(s, t) \rightarrow t$, the result of which subsumes the continuous-time CM (Lu & Song, 2024) as a single-particle estimate (Appendix H). We leave experiments for this to future work.

Diffusion GAN and Adversarial Consistency Distillation. Diffusion GAN (Xiao et al., 2021) parameterizes the generative distribution as $p_{s|t}^\theta(\mathbf{x}_s|\mathbf{x}_t) = \int q_{s|t}(\mathbf{x}_s|\mathbf{x}, \mathbf{x}_t) \delta(\mathbf{x} - \mathbf{G}_\theta(\mathbf{x}_t, \mathbf{z}, t)) p(\mathbf{z}) d\mathbf{z} d\mathbf{x}$ for s as a fixed decrement from t and $p(\mathbf{z})$ a noise distribution. It defines the interpolant $q_{s|t}(\mathbf{x}_s|\mathbf{x}, \mathbf{x}_t)$ as the DDPM posterior distribution, which is self-consistent (see Appendix G.2) and introduces randomness to the sampling process to match $q_t(\mathbf{x}|\mathbf{x}_t)$ instead of the marginal. Both Diffusion GAN and Adversarial Consistency Distillation (Sauer et al., 2025) use GAN objective, which shares similarity to MMD in that MMD is defined as an integral probability metric where the optimal discriminator is chosen in RKHS. This eliminates the need for explicit adversarial optimization of a neural-network discriminator.

Generative Moment Matching Network. GMMN (Li et al., 2015) directly applies MMD to train a generator $\mathbf{G}_\theta(\mathbf{z})$ where $\mathbf{z} \sim \mathcal{N}(0, I)$ to match the data distribution. It is a special case of IMM in that when $t = 1$ and $r(s, t) \equiv s = 0$ our loss reduces to naïve GMMN objective.

6. Related Works

Diffusion, Flow Matching, and stochastic interpolants. Diffusion models (Sohl-Dickstein et al., 2015; Song et al., 2020b; Ho et al., 2020; Kingma et al., 2021) and Flow Matching (Lipman et al., 2022; Liu et al., 2022) are widely used generative frameworks that learn a score or velocity field of a noising process from data into a simple prior. They have been scaled successfully for text-to-image (Rombach et al., 2022; Saharia et al., 2022; Podell et al., 2023; Chen et al., 2023; Esser et al., 2024) and text-to-video (Ho et al., 2022a; Blattmann et al., 2023; OpenAI, 2024) tasks.

Stochastic interpolants (Albergo et al., 2023; Albergo & Vanden-Eijnden, 2022) extend these ideas by explicitly defining a stochastic path between data and prior, then matching its velocity to facilitate distribution transfer. While IMM builds on top of the interpolant construction, it directly learns one-step mappings between any intermediate marginal distributions.

Diffusion distillation. To resolve diffusion models’ sampling inefficiency, recent methods (Salimans & Ho, 2022; Meng et al., 2023; Yin et al., 2024; Zhou et al., 2024; Luo et al., 2024a; Heek et al., 2024) focus on distilling one-step or few-step models from pre-trained diffusion models. Some approaches (Yin et al., 2024; Zhou et al., 2024) propose jointly optimizing two networks but the training requires careful tuning in practice and can lead to mode collapse (Yin et al., 2024). Another recent work (Salimans et al., 2024) explicitly matches the *first* moment of the data distribution available from pre-trained diffusion models. In contrast, our method implicitly matches *all* moments using MMD and can be trained from scratch with a single model.

Few-step generative models from scratch. Early one-step generative models primarily relied on GANs (Goodfellow et al., 2020; Karras et al., 2020; Brock, 2018) and MMD (Li et al., 2015; 2017) (or their combination) but scaling adversarial training remains challenging. Recent independent classes of few-step models, e.g. Consistency Models (CMs) (Song et al., 2023; Song & Dhariwal, 2023; Lu & Song, 2024), Consistency Trajectory Models (CTMs) (Kim et al., 2023; Heek et al., 2024) and Shortcut Models (SMs) (Frans et al., 2024) still face training instability and require specialized components (Lu & Song, 2024) (e.g., JVP for flash attention) or other special practices (e.g., high weight decay for SMs, combined LPIPS (Zhang et al., 2018) and GAN losses for CTMs, and special training schedules (Geng et al., 2024)) to remain stable. In contrast, our method trains stably with a single loss and achieves strong performance without special training practices.

7. Experiments

We evaluate IMM’s empirical performance (Section 7.1), training stability (Section 7.2), sampling choices (Section 7.3), scaling behavior (Section 7.4) and ablate our practical decisions (Section 7.5).

7.1. Image Generation

We present FID (Heusel et al., 2017) results for unconditional CIFAR-10 and class-conditional ImageNet-256×256 in Table 1 and 2. For CIFAR-10, we separate baselines into diffusion and flow models, distillation models, and few-step models from scratch. IMM belongs to the last category in which it achieves state-of-the-art performance of 1.98